

Sistema de Gestión de Biblioteca - Estructuras de Datos II

1. Justificación Técnica de las Estructuras de Datos

Para la solución del “Catálogo de biblioteca”, fue seleccionado e implementado desde cero tres estructuras de datos principales, se detalla la justificación matemática y algorítmica de cada elección en función de las operaciones más frecuentes:

A. Árbol B+

Propósito: Indexar y almacenar todos los libros del sistema utilizando el Código único como clave de búsqueda.

- **Justificación:** En un sistema de biblioteca, la operación más frecuente es la búsqueda de un libro por su identificador exacto, el ÁrbolB+ garantiza que las operaciones de búsqueda, inserción y actualización se realicen en un tiempo asintótico donde m es el orden del árbol.
- **Ventaja:** A diferencia de un BST tradicional o un AVL, el ÁrbolB+ almacena los datos reales únicamente en las hojas y mantiene un factor de ramificación alto, esto reduce drásticamente la altura del árbol, permitiendo accesos sumamente rápidos

B. Max Heap (Top 5 Libros Más Prestados)

- **Propósito:** Mantener un registro dinámico de los libros más populares según su atributo VecesPrestado.
- **Justificación:** Obtener los libros más solicitados requiere conocer los valores máximos, si se usara un arreglo normal, habría que ordenarlo completo en cada consulta, al utilizar un Max Heap, el libro más prestado siempre se mantiene en la raíz del árbol.
- **Eficiencia:** Cuando un libro se presta, su contador aumenta, el Heap simplemente aplica un método de Heapify-Up (Subir) o Reestructurar que toma $O(\log n)$, haciendo que la actualización estadística sea extremadamente eficiente sin penalizar el rendimiento del sistema.

C. Min Heap-Catálogo Ordenado por Título

- **Propósito:** Generar un listado completo de todos los libros ordenados alfabéticamente de la A a la Z.

- **Justificación:** El requerimiento exige poder listar el catálogo ordenado por Título, en lugar de implementar un algoritmo de ordenamiento tradicional sobre un arreglo desordenado cada vez que se pide el reporte, se insertan los libros en un Min Heap utilizando una función de comparación de cadenas.
- **Operación:** La raíz siempre contendrá el libro alfabéticamente menor, al extraer elementos sucesivamente, se obtiene el catálogo ordenado alfabéticamente de manera natural y óptima.

2. Explicación de la Implementación

El sistema fue desarrollado en C#:

1. **Cero Colecciones Nativas:** No se utilizó List<T>, Dictionary, ni PriorityQueue. Para manejar el crecimiento dinámico de datos auxiliares (fuera de las estructuras principales), se programó una clase propia llamada ArregloDinamico<T>, la cual duplica la capacidad de sus arreglos primitivos (T[]) mediante realojamiento de memoria cuando se alcanza el límite, logrando un costo amortizado de inserción de $O(1)$.
2. **Operación de Eliminación (Baja Lógica):** La eliminación estructural de nodos en un Árbol B+ requiere complejos algoritmos de Borrow o Merge para mantener la integridad del índice y la consistencia histórica de los préstamos, se implementó la Baja Lógica, la clase Libro posee un atributo booleano Eliminado, la función Eliminar marca este atributo como true, excluyendo al libro de las búsquedas y reportes en un tiempo de $O(\log n)$, sin desbalancear el árbol ni los Heaps.
3. **Sincronización por Referencia:** Al estar desarrollado en un lenguaje orientado a objetos, el ÁrbolB+ y los Heaps almacenan referencias al mismo espacio de memoria del objeto Libro, cuando se realiza un préstamo, se actualiza el objeto y se llama al método Reestructurar de los Heaps para que evalúen nuevamente a sus hijos y se acomoden en $O(n)$, manteniendo los Tops exactos.

Diagrama de Clases

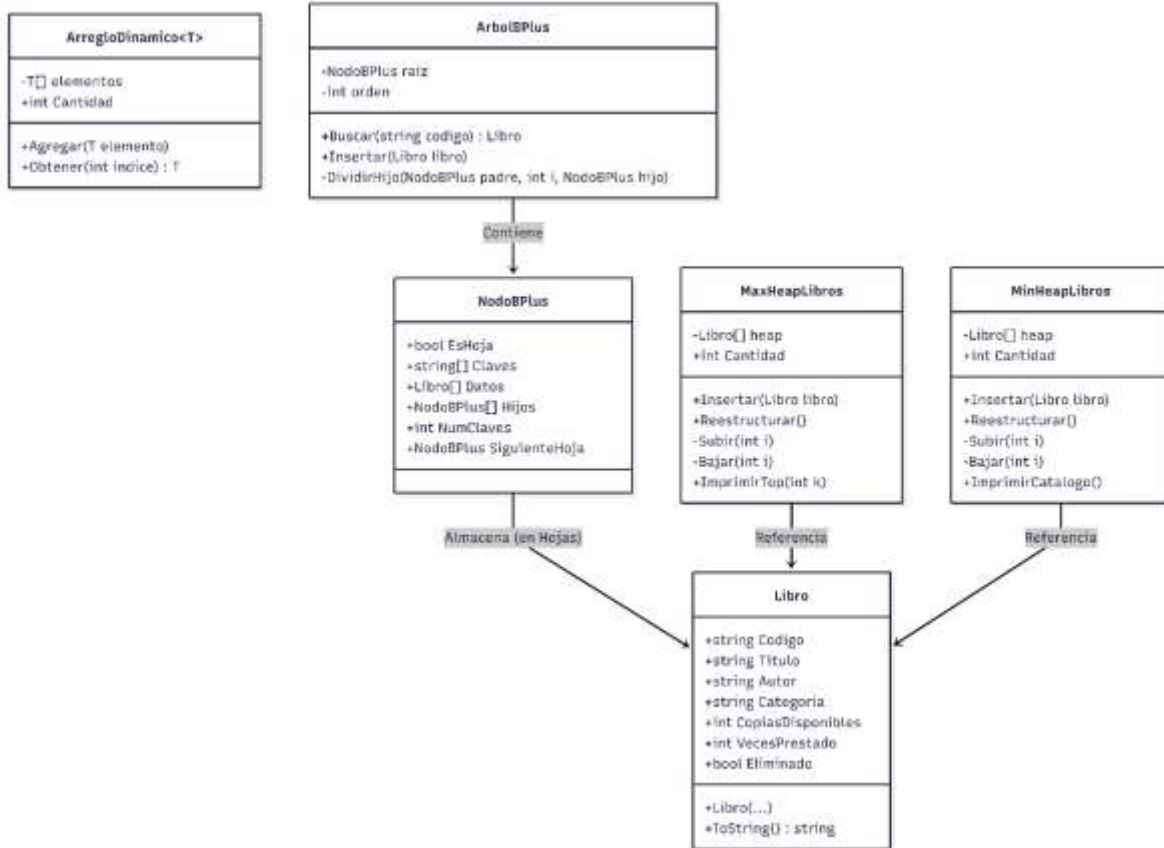


Diagrama de Flujo

